arXiv:2306.12303v2 [quant-ph] 10 Aug 2023

A New Initial Distribution for Quantum Generative Adversarial Networks to Load Probability Distributions

Yuichi Sano*[1], Ryosuke Koga[1], Masaya Abe[2], and Kei Nakagawa[2]

[1]Department of Nuclear Engineering, Kyoto University,
Nishikyo-ku, Kyoto 615-8540, Japan
[2]Nomura Asset Management Co.,Ltd.

August 11, 2023

Abstract

Quantum computers are gaining attention for their ability to solve certain problems faster than classical computers, and one example is the quantum expectation estimation algorithm that accelerates the widely-used Monte Carlo method in fields such as finance. A previous study has shown that quantum generative adversarial networks(qGANs), a quantum circuit version of generative adversarial networks(GANs), can generate the probability distribution necessary for the quantum expectation estimation algorithm in shallow quantum circuits. However, a previous study has also suggested that the convergence speed and accuracy of the generated distribution can vary greatly depending on the initial distribution of qGANs' generator.In particular, the effectiveness of using a normal distribution as the initial distribution has been claimed, but it requires a deep quantum circuit, which may lose the advantage of qGANs.

Therefore, in this study, we propose a novel method for generating an initial distribution that improves the learning efficiency of qGANs.Our method uses the classical process of *label replacement* to generate various probability distributions in shallow quantum circuits. We demonstrate that our proposed method can generate the log-normal distribution, which is pivotal in financial engineering, as well as the triangular distribution and the bimodal distribution, more efficiently than current methods.

Additionally, we show that the initial distribution proposed in our research is related to the problem of determining the initial weights for qGANs.

*sano.yuichi.77v@st.kyoto-u.ac.jp

1 Introduction

Quantum computers have attracted attention for their usefulness in solving specific problems faster than classical computers, and in recent years they have been the subject of very active research and development in various fields [1, 2]. One area where quantum computing could be particularly beneficial is in fields that rely heavily on Monte Carlo methods, such as financial engineering. Since quantum computers can utilize a fast expectation estimation algorithm, known as the quantum expectation estimation algorithm, it is expected that the algorithm can be used instead of the Monte Carlo method in classical computers to speed up expectation estimation [3–5].

However, it is known that current Noisy Intermediate-Scale Quantum (NISQ) devices under development are noisy due to the lack of error correction. Additionally, there are limits to the circuit depth, which corresponds to how many gates can be executed in a quantum circuit, and the number of qubits corresponding to a size of a computation [6, 7]. Therefore, there are many ongoing studies to improve algorithms to make existing quantum algorithms viable for NISQ devices [8, 9].

Executing the quantum expectation estimation algorithm needs the construction of a quantum circuit capable of loading a probability distribution into a quantum state. One method is the Grover-Rudolph state preparation [11], but it requires up to $O(2^n)$ gates and quantum circuit depth, making it difficult to use in NISQ devices. To address this issue, a method using quantum Generative Adversarial Networks(qGANs), which are quantum circuit versions of Generative Adversarial Networks(GANs [10]), was proposed in a previous research [8]. Previous research has shown that qGANs can be used to learn the parameters of quantum circuits to generate shallow quantum circuits that load probability distributions into quantum states. On the other hand, the same previous research suggested that the speed of learning and the accuracy of the generated distributions differed greatly depending on the initial distribution used for a qGANs' generator. In particular, the effectiveness of using the normal distribution as the initial distribution was argued, but using the normal distribution requires more depth of the quantum circuit. In the previous research, the circuit with a depth of "$d = 4$" could make the normal distribution, including errors, the initial distribution in the case of 3 qubits. However, as the number of qubits increases, a deeper quantum circuit is considered necessary to generate a normal distribution. Therefore, qGANs using the normal distribution for the initial distribution may lose the advantage of shallow circuit depth, a feature of circuit generation using qGANs.

We propose a method for creating initial distributions that can be implemented in shallow quantum circuit depth, enhancing the learning efficiency of qGANs.Specifically, we propose a method to generate various probability distributions even for circuits with gate depth $d = 1$, similar to the uniform distribution, by a classical process of *label replacement*. Since the only operations required to execute the proposed method can be performed on a classical computer, it can be efficiently implemented on a quantum computer, including NISQ

devices.We then propose a method to create an initial distribution "near" the target distribution using only single-qubit gates by *label replacement*. To confirm the effectiveness of our method, we conducted simulations to verify that the log-normal, triangular, and bimodal distributions investigated in previous studies can be generated more efficiently and accurately than previous initial distributions. We have shown by simulation that our method performs better than the uniform distribution consisting of single-qubit gates for all target distributions and better than the normal distribution for the log-normal distribution, which is essential in financial engineering. We also showed that the initial distribution proposed in this study is the initial weights of the generator and, consequently, can be implemented at depth $d = 0$.

2 Preliminaries

In this section, we outline the method of loading probability distributions into quantum states using qGANs, which is the basis of this study.Furthermore, we detail its practical application within the context of the quantum expectation estimation algorithm.

2.1 Quantum Expectation Estimation Algorithm

Monte Carlo-based expectation estimation using a classical computer requires $\mathcal{O}(1/\epsilon^2)$ number of simulations for the desired accuracy ϵ. In contrast, the quantum expectation estimation algorithm, which uses an amplitude estimation algorithm, is known to be able to estimate an expectation in $\mathcal{O}(1/\epsilon)$ oracle queries [3].

The quantum expectation estimation algorithm consists of two steps: (1) preparing a quantum state whose probability amplitude has an expected value of an estimation target and (2) estimating its amplitude. We show how to load the target expectation into a probability amplitude of a quantum state. First, we construct the following oracle that loads the target probability into the probability amplitude;

$$\mathcal{P}\left|0\right\rangle^{\otimes n} = \sum_{i}^{N} \sqrt{p_i}\left|i\right\rangle, \tag{1}$$

where i is each event, p_i is the probability of event i occurring, and n is the number of qubits such that the number of all events $N = 2^n$. For this quantum state, the probability of measuring $|1\rangle$ when the second register is measured in a computational basis is

$$\Pr[|1\rangle] = \sum_{i}^{N} f_i p_i \tag{2}$$

which is the expected value of the target. Therefore, an oracle $\mathcal{Q} = \mathcal{P}\mathcal{F}$ that combines these two oracles $\mathcal{P}$ and $\mathcal{F}$ is an oracle that loads the desired expected value into a probability amplitude of a quantum state.

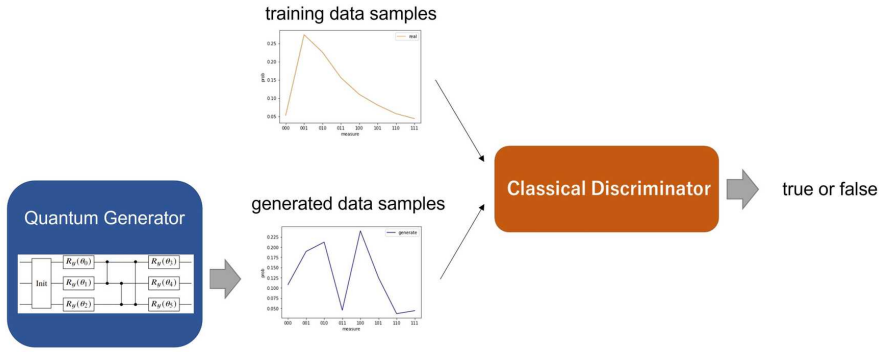


Figure 1: Loading Probability Distributions by qGANs

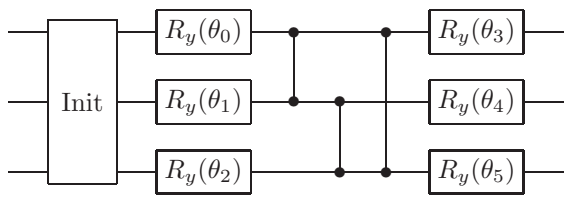


Figure 2: A quantum circuit with parameters that serve as a quantum generator. The circuit Init is the circuit that generates an initial distribution for learning.

We then show quantum amplitude estimation to estimate the probability amplitude using this oracle $\mathcal{Q}$ and the quantum phase estimation algorithm [12]. Recently, algorithms have been proposed to avoid the inverse quantum Fourier transform, which has a deep circuit [13, 14]. We simplify the notation of the quantum state after running the oracle $\mathcal{Q}$ as follows;

$$\mathcal{Q}\left|0\right\rangle^{\otimes(n+1)} = \cos\theta\left|\psi_0\right\rangle\left|0\right\rangle + \sin\theta\left|\psi_1\right\rangle\left|1\right\rangle. \tag{3}$$

The probability amplitude we want to find is $\sin\theta$. In addition to the oracle $\mathcal{Q}$, oracles $\mathcal{S}_0 = I - 2\left|0\right\rangle\left\langle 0\right|^{\otimes(n+1)}$, and $\mathcal{S}_{\psi_0} = I - 2\left|\psi_0\right\rangle\left\langle\psi_0\right| \otimes \left|0\right\rangle\left\langle 0\right|$ are combined to form a oracle $\mathcal{U} = \mathcal{Q}\mathcal{S}_0\mathcal{Q}^\dagger\mathcal{S}_{\psi_0}$. Since the phase of the oracle $\mathcal{U}$ is $\pm e^{i2\theta}$, the quantum phase estimation algorithm can estimate θ and $\sin\theta$ with an error ϵ by querying $\mathcal{O}(1/\epsilon)$ times for the oracle $\mathcal{Q}$. As mentioned above, this $\sin\theta$ is the desired expected value in the quantum expectation estimation algorithm, so the computational complexity is improved compared to using the Monte Carlo method.

2.2 Loading Probability Distributions by qGANs

In order to run the quantum expectation estimation algorithm, it is necessary to construct the oracle $\mathcal{P}$ that loads a probability distribution into a probability

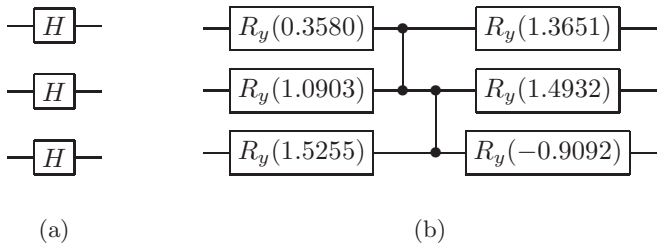


Figure 3: Circuits to generate initial distributions. (a) Circuit to generate the uniform distribution. (b) Circuit to generate the normal distribution with mean $\mu = 1$ and variance $\sigma = 1$(for log-normal distribution).

amplitude. It is known that using the method Grover-Rudolph state preparation [11] for this purpose requires the use of up to $\mathcal{O}(2^n)$ gates, which leads to a deep circuit depth. Therefore, to construct oracles that generate probability distributions with shallower quantum circuits, the method using qGANs was proposed in previous research [8].

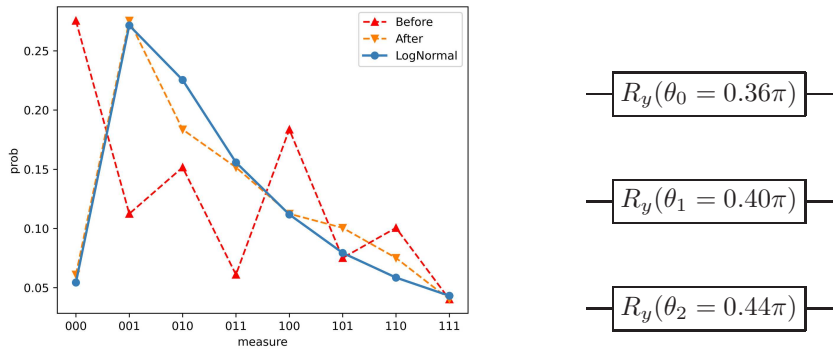
Generative Adversarial Networks (GANs) are generative models used to create false data nearly indistinguishable from the true data they are based. In GANs, the generator, which generates false data, and the discriminator, which discriminates between false and true data, learn from each other so that, eventually, an adequately trained generator will output false data indistinguishable from true data. The qGANs are an extension to a generator by replacing the generator with a quantum circuit, as shown in Figure 1. In previous research, a method was proposed to create a quantum circuit that prepares a quantum state with probability amplitude indistinguishable from the target probability distribution by providing data sampled from the target probability distribution as true data for the qGANs. The quantum generator is a quantum circuit with parameters such that the rotation angle θ_i of the R_y gate is varied as a "weight", as shown in Figure 2. The $R_y(\theta_i)$ gates represent the rotation of the y-axis of the Bloch sphere and correspond to the following unitary operator;

$$R_y(\theta_i) = \begin{pmatrix} \cos\frac{\theta_i}{2} & -\sin\frac{\theta_i}{2} \\ \sin\frac{\theta_i}{2} & \cos\frac{\theta_i}{2} \end{pmatrix}. \tag{4}$$

3 Method

Our purpose is to introduce a method for creating initial distributions that uses only single-qubit gates and is more efficient in learning qGANs.

For this purpose, we propose *label replacement* that can create various initial distributions from a probability distribution that can be generated from single-qubit gates. Specifically, *label replacement* reproduces various initial distributions without increasing the number of quantum gates by appropriately replacing the measurement results on a computational basis. For example, by



(a) (b)

Figure 4: Transformation of the initial distribution by *label replacement*. (a) "Before" is the distribution before *label replacement*, "After" is the distribution after *label replacement*, and "LogNormal" is the distribution that approximates the target log-normal distribution(mean $\mu = 1$ and variance $\sigma = 1$). (b) The specific quantum gate of the quantum circuit Init loads the distribution before *label replacement*.

replacing Figure 4a, we reproduce an initial distribution close to the target log-normal distribution. The circuit Init for generating the distribution before the replacement of Figure 4a can be implemented with a gate of depth $d = 1$ composed by Figure 4b. If one were to consider replacing the measurement results with only operations on the quantum circuit, the replacement would require two-qubit gates, and it is impossible to implement at a depth of $d = 1$. In contrast, *label replacement* can be implemented by a classical process because it transforms a random variable that determines what value corresponds to the output of a measurement result. Therefore, this method is easy to implement in any quantum circuit, including NISQ devices.

We then show how to create an initial distribution that reflects knowledge using *label replacement*. To prepare an initial distribution that is "near" to the target probability distribution, we first sort the target probability distribution in order of increasing probability. Next, we determine the angle of R_y so that the sum of the probabilities of the top half corresponds to the measured probability of $|0\rangle$. Then we determine the angle of R_y so that the ratio of the sum of its further top half probabilities corresponds to the measurement probability of $|0\rangle$. We repeat the above procedure to determine the angle of R_y. Using this procedure, we can always generate an initial distribution such that the largest probability in the target probability distribution coincides with the largest probability in the probability distribution created by *label replacement*. We present the exact algorithm in Algorithm 1.

Algorithm 1 Fitting Probability Distributions

Input: $\{p_n\}_n$ ▷ Input is a target probability distribution.
Output: θ_{target} ▷ Output is the rotation angle of the R_y gates.
$\{q_n\}_n \leftarrow \mathrm{Sort}(\{p_n\}_n)(\therefore q_i \geq q_{i-1})$ ▷ Sort p_n.
$u_1 \leftarrow 1$
$\theta_{target} = []$
for $i = 1$ **to** $\log(n)$ **do** ▷ Calculate the rotation angle θ_i.
  $u_i = \Sigma_{j=1}^{n/2^i} q_j$
  $r_i = u_i / u_{i-1}$
  $\theta_i = 2 \arccos(r_i) \quad (0 \leq \theta_i \leq \frac{\pi}{2})$
  $\theta_{target}.\mathrm{append}(\theta_i)$ ▷ Add the obtained rotation angle to the list θ_{target}.
end for

4 Simulation Test

The performance of our initial distributions was tested by simulating a quantum computer with a classical computer.

4.1 Simulation Setup

Our simulation setup was the same as previous research [8] except that we added our initial distribution. We also used log-normal, triangular, and bimodal distributions to evaluate performance. The log-normal distribution had a mean $\mu = 1$ and a standard deviation $\sigma = 1$, the triangular distribution had a lower limit $l = 0$, an upper limit $u = 7$, and mode $m = 2$, and the bimodal distribution consisted of two superimposed Gaussian distributions with $\mu_1 = 0.5$, $\sigma_1 = 1$ and $\mu_2 = 3.5$, $\sigma_2 = 0.5$. The quantum generator of the qGANs consists of an initial distribution circuit *init* and gates with parameters as shown in Figure 1. The circuit with parameters for the latter part of the quantum generator consists of a repeating quantum circuit with one set of combinations of a section of R_y gates with the rotation angle θ_i as weights and a section of CZ gates to affect multiple qubits. The iterations of these gates correspond to the number of parameters, with $N_p = 3(k+1)$ number of parameters for k number of iterations. The initial weights of the generators were randomly sampled values from a uniform distribution in the range $[-10^{-1}, 10^{-1}]$. The classical discriminator was implemented using Pytorch [15]. The discriminator consists of a 50-node input layer, a 20-node hidden layer, and a 1-node output layer. The AMSGRAD [16] was used for training, with a learning rate of $lr = 10^{-4}$ and coefficients for calculating moments of a gradient of $\beta_1 = 0.7$ and $\beta_2 = 0.99$. The parameter shift rule [17, 18] was used to calculate the gradient of the quantum generator, and the number of samples was set to 8000 times. The following non-saturating loss function [19] is used for a loss function:

$$\mathcal{L}_G(\phi, \theta) = -\mathbb{E}_{z \sim p_{\mathrm{prior}}} \left[\log D_\phi(G_\theta(z)) \right], \tag{5}$$

$$\mathcal{L}_D(\phi,\theta) \quad = \quad \mathbb{E}_{x\sim p_{\text{real}}}\left[\log D_\phi(x)\right] + \mathbb{E}_{z\sim p_{\text{prior}}}\left[\log(1-D_\phi(G_\theta(z)))\right], \quad (6)$$

where p_{prior} denote the true probability distribution of z, p_{real} denote the probability distribution obtained from the data, $\mathcal{L}_D$ is the loss function of the discriminator and $\mathcal{L}_G$ is the loss function of the generator.

As training data, we used samples from each target probability distribution, sampled 2000 times, and discretized by 8 points. These data were divided into mini-batches of batch size 2000, and with the addition of 2000 samples measured from the quantum generators, we trained the qGANs. The training was performed up to 2000 epochs, with ten training runs each for each initial distribution and each target probability distribution to ensure robustness.

In this study, the initial distribution was our initial distribution, the uniform distribution, and the normal distribution with a mean and standard deviation of the target probability distribution. Let $\theta_{target} = [\theta_0, \theta_1, \theta_2]$ be the parameters that make up our initial distribution, as shown in Figure 4b. The parameters of our initial distribution are

$$\theta_{lognormal} = [0.36\pi, 0.40\pi, 0.44\pi] \quad (7)$$

for the log-normal distribution,

$$\theta_{triangular} = [0.29\pi, 0.43\pi, 0.48\pi] \quad (8)$$

for the triangular distribution, and

$$\theta_{bimodal} = [0.21\pi, 0.44\pi, 0.48\pi] \quad (9)$$

for the bimodal distribution. Also, the permutation of each measurement outcome is

$$\sigma_{lognormal} = \begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 3 & 0 & 4 & 2 & 1 & 6 & 5 & 7 \end{bmatrix} \quad (10)$$

for the log-normal distribution,

$$\sigma_{triangular} = \begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 3 & 6 & 0 & 4 & 2 & 1 & 5 & 7 \end{bmatrix} \quad (11)$$

for the triangular distribution, and

$$\sigma_{bimodal} = \begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 2 & 6 & 1 & 0 & 4 & 5 & 3 & 7 \end{bmatrix} \quad (12)$$

for the bimodal distribution. We tested for the case $k = 1, 2, 3$ as the number of iterations of the quantum circuit.

For the simulation of a quantum computer, we used qiskit [20], a quantum computation library provided by IBM.

We use relative entropy to measures to evaluate the training performance. Relative entropy is a type of statistical distance that measures the closeness of the discrete probability distribution $P(x)$ and $Q(x)$ and is given by

$$H(P|Q) := \sum_{x\in X} P(x)\log\left(\frac{P(x)}{Q(x)}\right). \quad (13)$$

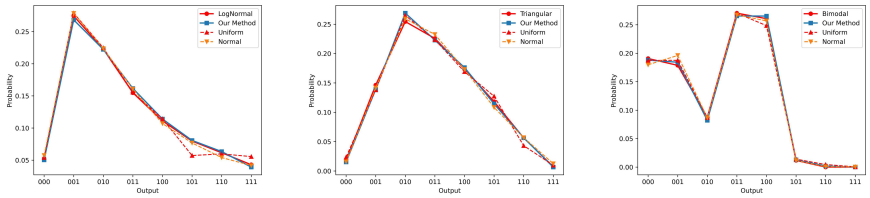
Relative entropy is non-negative, and the probability distributions $P(x)$ and $Q(x)$ are equal if and only if the value is zero.

4.2 Result

Table 1: Relative entropy of the result of learning the target distribution for ten times: The table shows the initial distributions(Init), the repeat(k), the mean (μ), the standard deviation (σ), and the minimum(Min).

Data	Init	k	μ	σ	Min
log-normal	our method	1	0.00190	0.000290	0.00134
		2	0.00163	0.000405	0.00092
		3	0.00163	0.000514	0.00086
	uniform	1	0.14514	0.013234	0.12992
		2	0.07810	0.043166	0.03962
		3	0.07702	0.052596	0.00706
	normal	1	0.00414	0.005282	0.00129
		2	0.01260	0.009894	0.00119
		3	0.01320	0.018007	0.00097
triangular	our method	1	0.03962	0.007225	0.03078
		2	0.03931	0.011143	0.00744
		3	0.02480	0.017297	0.00165
	uniform	1	0.36796	0.129145	0.06295
		2	0.08249	0.068867	0.01054
		3	0.19945	0.264557	0.00274
	normal	1	0.00212	0.000903	0.00136
		2	0.00289	0.003174	0.00104
		3	0.00288	0.001490	0.00121
bimodal	our method	1	0.17156	0.041284	0.10309
		2	0.11749	0.077465	0.00474
		3	0.09782	0.067075	0.00194
	uniform	1	0.34285	0.015677	0.31533
		2	0.16386	0.060814	0.05680
		3	0.04484	0.039178	0.01275
	normal	1	0.00953	0.001066	0.00786
		2	0.00846	0.002848	0.00100
		3	0.00863	0.003180	0.00191

The results of the simulation are summarized in Table 1. From Table 1, our method is superior to the uniform distribution as initial distribution in both mean and minimum values of relative entropy, except for the mean value of $k = 3$ for the bimodal distribution. Our method and the uniform distribution consist of single-qubit gates, which means that our method is a better initial distribution for the same cost. Also, our method is inferior in the mean value of relative entropy but equally suitable for the minimum value compared to the



(a) log-normal (b) triangular (c) bimodal

Figure 5: Comparison of generated probability distributions. The lowest relative entropy for our proposed method ("Our Method"), the uniform distribution ("Uniform"), and the normal distribution ("Normal") with $k = 3$.

case where the normal distribution is an initial distribution. In particular, when the log-normal distribution is the target distribution, our method outperforms the normal distribution as the initial distribution in both mean and minimum values, indicating the superiority of our method. Our method may be better in NISQ devices because the normal distribution requires two-qubit gates, which is expensive due to the high noise content in NISQ devices, and the depth is deeper than our method. Our method is also robust to the qubit number scale because it is always "$d = 1$", whereas normal distributions may lose the advantage of circuit generation using qGANs if the quantum circuit making the normal distribution is deeper for more than four qubits.

As a legend, the probability distribution with the lowest relative entropy generated from each initial distribution is shown in Figure 5.

4.3 Relationship with Initial Weight

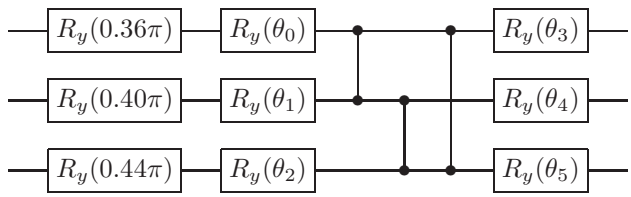
The quantum circuit that generates our initial distributions is realized by $R_y(\theta)$ gates corresponding to the rotation of the Bloch sphere by an angle θ with the y-axis of the sphere, as shown in Figure 4b. In addition, the parameter corresponding to the quantum generator weights is the rotation angle θ_i of the $R_y(\theta_i)$ gate, as shown in Figure 2. Let the rotation angles be θ_1 and θ_2, respectively, then

$$R_y(\theta_1) \cdot R_y(\theta_2) = R_y(\theta_1 + \theta_2). \tag{14}$$

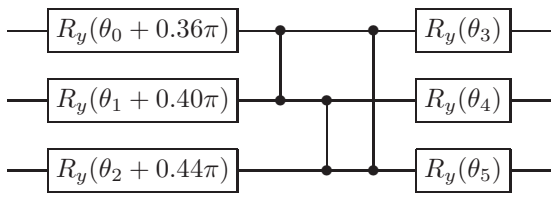
In other words, R_y gates can be composited with each other, and after compositing, they correspond to a rotational gate R_y with the sum of their respective angles as the rotation angle. Hence, our initial distribution can be understood as the initial weights of the quantum generators, as shown in Figure 6.

When the initial distribution is the uniform distribution, the quantum circuit $Init$ consists of three Hadamard gates $H^{\otimes 3}$, as shown in Figure 3a, and the operation of a Hadamard gate H on the initial state $|0\rangle$ is

$$H\,|0\rangle = \frac{1}{\sqrt{2}}\,|0\rangle + \frac{1}{\sqrt{2}}\,|1\rangle\,, \tag{15}$$



(a)



(b)

Figure 6: Relationship with Initial Weight. (a) Quantum generators with our proposed distribution as the initial distribution. (b) Quantum generators with our proposed distribution as initial weights.

and the operation of $R_y(\frac{\pi}{2})$ on the initial state $|0\rangle$ is

$$R_y(\frac{\pi}{2})\,|0\rangle = \frac{1}{\sqrt{2}}\,|0\rangle + \frac{1}{\sqrt{2}}\,|1\rangle\,, \tag{16}$$

so these two operations are the same for the initial state $|0\rangle$. Therefore, if $R_y(\frac{\pi}{2})$ is used instead of the Hadamard gate H to create the uniform distribution, this can be composed into the generator parameters as well, corresponding to the fact that the initial weights were chosen as $\frac{\pi}{2}$. The same interpretation partially holds for the normal distribution since the last column of the circuit in Figure 3b is the R_y gate.

In summary, the initial distribution of our method and the uniform distribution can be interpreted as the initial weights of the quantum generator and, simultaneously, as the initial distribution that can be implemented at depth $d = 0$.

5 Conclusion

We proposed the new method for creating initial distributions by combining single-qubit gates and the classical methods of *label replacement* for the qGANs. Additionally, we proposed new initial distributions that would improve the accuracy of quantum circuit generation using qGANs by using this method and

reflecting knowledge of the target probability distributions. We then tested whether our proposed initial distributions performed better than existing initial distributions by simulation using a classical computer. The results indicated that our method consistently outperformed uniform distributions composed of single-qubit gates across all target probability distributions. Moreover, it surpassed the more costly normal distributions when focusing on log-normal distributions. In the previous research, a relatively low-cost implementation of the normal distributions was proposed only for the 3-qubit circuit, and we consider that our method is more advantageous for 4-qubit and above. In addition, deeper quantum circuits may be required to prepare the normal distribution for four or more qubits, potentially eliminating the benefit of circuit generation using qGANs. In contrast, our initial distribution can consistently be implemented with "d = 0" and is robust to the scale of the number of qubits, which is also superior.

We also showed that the proposed and existing initial distributions are composable with the generator weights and correspond to the initial weights. Consequently, we have shown that our initial distributions, as well as uniform distributions, can be implemented at depth $d = 0$.

Our future works are to examine whether this method maintains its efficacy for probability distributions beyond the log-normal, triangular, and bimodal distributions. We also intend to assess its performance as the number of qubits increases. In order to achieve the desired level of accuracy,it is also necessary to evaluate the impact of the number of measurements and the number of parameters (weights) on accuracy.

Acknowledgment

YS and RK would like to thank Takayuki Miyadera for the many helpful comments. YS would like to thank Ikko Hamamura for the technical advice about coding. This work was supported by JST SPRING Grant Number JPMJSP2110 and JSPS KAKENHI Grant Numbers JP23KJ1178.

Competing Interests

All authors declare no financial or non-financial competing interests.

Data Availability

The datasets used and analysed during the current study available from the corresponding author on reasonable request.

Code Availability

The underlying code for this study is not publicly available but may be made available to qualified researchers on reasonable request from the corresponding author.

References

[1] Shor, P.W., "Algorithms for quantum computation: discrete logarithms and factoring", *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pp. 124–134, 1994.

[2] Grover, L. K., "A fast quantum mechanical algorithm for database search", *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, pp. 212-–219, 1996.

[3] Rebentrost,P., Gupt,B., and T.R. Bromley, "Quantum computational finance: Monte Carlo pricing of financial derivatives", *Phys. Rev. A*, **98**, 022321, 2018.

[4] Egger, D.J., Garcia-Gutierrez, R., Mestre, J., and Woerner, S., "Credit Risk Analysis Using Quantum Computers", *IEEE Transactions on Computers*, **70**, pp. 2136–2145, 2021.

[5] Orús, R., Mugel, S., and Lizaso, E., "Quantum computing for finance: Overview and prospects", *Reviews in Physics*, **4**, 100028, 2019.

[6] Arute, F., Arya, K., Babbush, R., Bacon, D., Bardin, J. C. et al., "Quantum supremacy using a programmable' superconducting processor", *Nature*, **574**, pp. 505–510, 2019.

[7] Bharti, K., Cervera-Lierta, A., Kyaw, T. H., Haug, T., Alperin-Lea, S. et al., "Noisy intermediate-scale quantum algorithms", *Rev. Mod. Phys.*, **94**, 015004, 2022.

[8] Zoufal, C., Lucchi, A., and Woerner, S., "Quantum generative adversarial networks for learning and loading random distributions", *npj Quantum Inf*, **5**, 103, (2019).

[9] Peruzzo, A., McClean, J., Shadbolt, P., Yung, M. H., Zhou, X. Q. et al., "A variational eigenvalue solver on a photonic quantum processor", *Nature communications*, **5**, pp. 1–7, 2014.

[10] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D. et al., "Generative Adversarial Nets", *Advances in Neural Information Processing Systems*, **27**, 2014.

[11] Grover, L. K., and Rudolph, T., "Creating superpositions that correspond to efficiently integrable probability distributions", arXiv preprint quant-ph/0208112, 2002.

[12] Nielsen, M. A., and Chuang, I., "Quantum computation and quantum information", *Cambridge University Press*, Cambridge, 2002.

[13] Suzuki, Y., Uno, S., Raymond, R., Tanaka, T., Onodera, T., and Yamamoto, N., "Amplitude estimation without phase estimation", *Quantum Inf Process*, **19**, 75, 2020.

[14] Grinko, D., J. Gacon, Zoufal, C., and Woerner, S., "Iterative quantum amplitude estimation", *npj Quantum Inf*, bf 7, 52, 2021.

[15] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J. et al., "Pytorch: An imperative style, high-performance deep learning library", *Advances in neural information processing systems 32*, 2019.

[16] Reddi, S.J., Kale, S., and Kumar, S., "On the Convergence of Adam and Beyond", *International Conference on Learning Representations*, 2018.

[17] Mitarai, K., Negoro, M., Kitagawa, M., and Fujii, K. "Quantumcircuitlearning", *Phys. Rev. A*, **98**, 032309, 2018.

[18] Zeng, J., Wu, Y., Liu, J. G., Wang, L., and Hu, J., "Learning and inference on generative adversarial quantum circuits", *Phys. Rev. A*, **99**, 052306, 2018.

[19] Fedus, W., Rosca, M., Lakshminarayanan, B., Dai, A. M., Mohamed, S., and Goodfellow, I., "Many paths to equilibrium: GANs do not need to decrease a divergence at every step", *International Conferenceon Learning Representations*, 2018.

[20] Qiskit contributors, "Qiskit: An Open-source Framework for Quantum Computing", 2023.